

Fișă de lucru 2 – Subprograme în Python: Scriere și depanare (Mutabilitate)

Mică recapitulare: Modificarea listelor

Deoarece listele sunt **mutabile**, atunci când trimitem o listă ca parametru într-un subprogram, o putem modifica direct (in-place), iar schimbările vor fi vizibile și în afara funcției. Cele mai folosite operații sunt:

- **Modificarea prin index:** `lista[0] = 10` (schimbă valoarea primului element)
- **Adăugarea la final:** `lista.append(5)` (adaugă numărul 5 la sfârșitul listei)

Atenție capcană! Dacă atribuim variabilei o listă complet nouă folosind `lista = [...]` în interiorul funcției, legătura cu lista originală se rupe, iar modificarea **nu** se va vedea în exterior.

Exercițiul 1. Completează subprogramul.

Vrem să adăugăm un nou vizitator (numele "Ana") la finalul listei noastre de vizitatori, folosind un subprogram. Completează linia lipsă astfel încât, după apel, lista să se modifice corect.

```
def adauga_vizitator(lista_nume):  
    # Completeaza subprogramul de mai jos  
    _____  
  
vizitatori = ["Mihai", "Elena"]  
adauga_vizitator(vizitatori)  
  
# Trebuie sa afiseze ['Mihai', 'Elena', 'Ana']  
print("Vizitatori totali:", vizitatori)
```

Răspuns:

Exercițiul 2. Găsește și corectează greșeala.

Andrei vrea să își actualizeze lista de note `[7, 8, 9]` folosind un subprogram. El trebuie să facă două lucruri: să schimbe prima notă (să devină 10) și să adauge o notă nouă (nota 9) la finalul listei. A scris codul de mai jos, dar pe ecran se afișează tot lista veche.

Explicație: Atunci când scrii `lista = [...]` în interiorul funcției, legătura cu lista originală se rupe! Python va crea o listă complet nouă, valabilă doar local.

Cerință: Corectează codul de mai jos astfel încât să se afișeze `[10, 8, 9, 9]`, modificând DOAR interiorul funcției. Folosește operațiile descrise în recapitulare.

```
notele_mele = [7, 8, 9]

def actualizeaza_note(lista):
    lista = [10, 8, 9, 9] # <--- Aici este greșeala!

actualizeaza_note(notele_mele)
print(notele_mele) # Afiseaza tot [7, 8, 9]
```

Rezolvarea ta (Codul corect pentru funcție):

Exercițiul 3. Scriere de cod de la zero.

Scrie un subprogram numit `aplica_reducere`, care nu returnează nimic (este `void`).

Acesta primește ca parametru o listă de prețuri (numere reale). Subprogramul va parcurge lista și va modifica direct elementele: **va scădea 10 din fiecare preț mai mare de 50.**

Exemplu de utilizare:

```
preturi_magazin = [40, 100, 60, 20]
aplica_reducere(preturi_magazin)
print(preturi_magazin) # Va afisa: [40, 90, 50, 20]
```

Rezolvare (Scrie întregul subprogram aici):

Bibliografie:

<https://www.geeksforgeeks.org/python/mutable-vs-immutable-objects-in-python/>

<https://realpython.com/python-mutable-vs-immutable-types/>